

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Name	Course Code
Data Structures	CSA03

NAME: S. BHUVANESH

REG NO: 192521004

Question 15

Graph represented as matrix. Write pseudocode for Floyd–Warshall with negative cycle detection.

Constraints:

- Detect negative cycles
- Update distances

1.AIM

To write pseudocode for the **Floyd–Warshall algorithm** using a graph represented as a matrix, update the shortest distances between all pairs of vertices, and detect negative cycles.

Input

- n = number of vertices
- $\text{dist}[n][n]$ = adjacency/distance matrix
- INF = infinity for no direct edge

1. PROBLEM OVERVIEW

The Floyd–Warshall algorithm computes shortest-path distances between every pair of vertices in a weighted directed graph represented as an adjacency matrix. It works correctly with negative edge weights as long as the graph contains no negative-weight cycle. This task extends the classic algorithm to (1) update the distance matrix using the standard triple-loop relaxation, and (2) detect the presence of any negative-weight cycle by inspecting the diagonal of the final distance matrix, and additionally identify which vertices are affected by such a cycle.

2. INPUT REPRESENTATION

- Graph G given as an $n \times n$ adjacency matrix W , where $W[i][j]$ = weight of edge $i \rightarrow j$.
- $W[i][i] = 0$ for all i (distance from a vertex to itself, before any negative cycle is found).
- $W[i][j] = \infty$ (a large sentinel value, e.g., INF) if no direct edge exists from i to j .

- n = number of vertices in the graph.

3. PSEUDOCODE

```
1. for  $k \leftarrow 0$  to  $n-1$  do
2.   for  $i \leftarrow 0$  to  $n-1$  do
3.     for  $j \leftarrow 0$  to  $n-1$  do

4.       if  $\text{dist}[i][k] \neq \text{INF}$  AND  $\text{dist}[k][j] \neq \text{INF}$  then
5.          $\text{newDistance} \leftarrow \text{dist}[i][k] + \text{dist}[k][j]$ 

6.         if  $\text{newDistance} < \text{dist}[i][j]$  then
7.            $\text{dist}[i][j] \leftarrow \text{newDistance}$ 
8.         end if

9.       end if

10.    end for
11.  end for
12. end for

13. // Negative cycle detection
14. for  $i \leftarrow 0$  to  $n-1$  do
15.   if  $\text{dist}[i][i] < 0$  then
16.     print "Negative cycle detected"
17.     return
18.   end if
19. end for
```

20. print "No negative cycle"

21. print diste, affected

4. EXPLANATION OF KEY STEPS

Step(s)	Purpose
1–7	Initialize D with direct edge weights from W; $D[i][i] = 0$ and unreachable pairs = INF.
8–18	Standard Floyd–Warshall relaxation: for every pair (i, j), check if routing through an intermediate vertex k gives a shorter path, and update $D[i][j]$ accordingly (Constraint 2).
19–26	After all n relaxation rounds, if any diagonal entry $D[i][i]$ has become negative, vertex i lies on a negative-weight cycle (Constraint 1).
27–42	Negative cycles can make shortest paths to other vertices undefined (arbitrarily negative). This propagation step marks every vertex reachable from a negative-cycle vertex as “affected” and sets their distance to $-\infty$ so callers do not treat D as a valid finite shortest-path matrix for those pairs.
43	Return the (possibly partially invalid) distance matrix, a flag indicating whether a negative cycle exists, and the list of affected vertices.

5. COMPLEXITY ANALYSIS

Aspect	Complexity
Time — core relaxation (lines 9–18)	$O(n^3)$ — three nested loops over n vertices
Time — negative cycle detection (lines 21–26)	$O(n)$ — single scan of the diagonal
Time — affected-vertex propagation (lines 30–41)	$O(n^3)$ worst case — comparable to the main relaxation
Overall time complexity	$O(n^3)$
Space complexity	$O(n^2)$ — for matrices D , $Next$ (optional), and $O(n)$ for the affected array

6. CORRECTNESS NOTES

- The algorithm correctly computes all-pairs shortest paths as long as the graph has no negative-weight cycle reachable between the pair being queried.
- A negative cycle is detectable only after all n rounds of relaxation are complete, because $D[i][i]$ can only become negative once a full cycle of negative total weight has been considered as a path from i back to i .
- Constraint 1 (detection) is satisfied by the diagonal check; Constraint 2 (distance update) is satisfied by the standard triple-nested relaxation, which this pseudocode preserves without modification so existing correctness proofs for Floyd–Warshall still apply.

- If the graph is known to be free of negative cycles, lines 19–42 can be skipped entirely, reducing the algorithm to the classical Floyd–Warshall shortest-path computation.

7.CONCLUSION

The Floyd–Warshall algorithm finds shortest paths between every pair of vertices using matrix representation. The distance matrix is updated through each possible intermediate vertex, and a negative cycle is detected by checking whether any diagonal value $\text{dist}[i][i]$ becomes negative.